COGNIZANT DIGITAL NURTURE PROGRAM WEEK 3 TASK

1)Control Strucutres :

Query :

```
// created a table for customers
CREATE TABLE Customers (
    CustomerID NUMBER PRIMARY KEY,
    Name VARCHAR2(50),
    Age NUMBER,
    Balance NUMBER,
    IsVIP VARCHAR2(5)
);

// created a table for loans
CREATE TABLE Loans (
    LoanID NUMBER PRIMARY KEY,
    CustomerID NUMBER,
    InterestRate NUMBER,
    DueDate DATE,
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

// inserting customer records
INSERT INTO Customers VALUES (101,'John',65,15000,'FALSE');
INSERT INTO Customers VALUES (102,'Alice',45,8000,'FALSE');
INSERT INTO Customers VALUES (103,'David',70,25000,'FALSE');
COMMIT;
```

```
// insertinf loan records
INSERT INTO Loans VALUES (1,101,10,DATE '2026-07-10');
INSERT INTO Loans VALUES (2,102,12,DATE '2026-07-20');
INSERT INTO Loans VALUES (3,103,9,DATE '2026-09-15');
COMMIT;

// main query
BEGIN
    FOR cust IN (SELECT CustomerID, Age FROM Customers) LOOP
        IF cust.Age > 60 THEN
            UPDATE Loans
            SET InterestRate = InterestRate - 1
            WHERE CustomerID = cust.CustomerID;
        END IF;
    END LOOP;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Loan interest rates updated successfully.');
END;
/
```

```
// main query for customers
BEGIN
  FOR cust IN (SELECT CustomerID, Balance FROM Customers) LOOP
      IF cust.Balance > 10000 THEN
          UPDATE Customers
          SET IsVIP = 'TRUE'
          WHERE CustomerID = cust.CustomerID;
      END IF;
  END LOOP;

  COMMIT;

  DBMS_OUTPUT.PUT_LINE('VIP status updated successfully.');
END;
/
```

Output :

STDIN

```
SELECT * FROM Loans;
```

Output:

```
    LOANID CUSTOMERID INTERESTRATE DUEDATE
---------- ---------- ------------ --------------------
         1        101            9 10-JUL-26
         2        102           12 20-JUL-26
         3        103            8 15-SEP-26
```

STDIN

```
SELECT * FROM Customers;
```

Output:

```
CUSTOMERID NAME                                          AGE
---------- ------------------------------------------------ ----------
   BALANCE ISVIP
---------- -----
         101 John                                          65
       15000 TRUE

         102 Alice                                         45
        8000 FALSE

         103 David                                         70
       25000 TRUE
```

2) Stored Procedures :

Query :

```
// created a table named accounts
CREATE TABLE Accounts (
    AccountID NUMBER PRIMARY KEY,
    CustomerName VARCHAR2(50),
    AccountType VARCHAR2(20),
    Balance NUMBER
);

// inserted details of accounts
INSERT INTO Accounts VALUES (101,'John','Savings',10000);
INSERT INTO Accounts VALUES (102,'Alice','Savings',15000);
INSERT INTO Accounts VALUES (103,'David','Current',20000);
COMMIT;

// main query
CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest
IS
BEGIN
    UPDATE Accounts
    SET Balance = Balance + (Balance * 0.01)
    WHERE AccountType = 'Savings';

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Monthly interest processed successfully.');
END;
```

```
//created tables for employees
CREATE TABLE Employees (
    EmployeeID NUMBER PRIMARY KEY,
    EmployeeName VARCHAR2(50),
    Department VARCHAR2(30),
    Salary NUMBER
);
// inserted data of employees
INSERT INTO Employees VALUES(1,'Rahul','IT',50000);
INSERT INTO Employees VALUES(2,'Priya','HR',40000);
INSERT INTO Employees VALUES(3,'Amit','IT',60000);
COMMIT;

// main query for employees
CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus(
    p_department IN VARCHAR2,
    p_bonus IN NUMBER
)
IS
BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * p_bonus / 100)
    WHERE Department = p_department;

    COMMIT;


    DBMS_OUTPUT.PUT_LINE('Employee bonus updated successfully.');
END;
/
BEGIN
  UpdateEmployeeBonus('IT',10);
END;
/
```

```
// TransferFunds
CREATE OR REPLACE PROCEDURE TransferFunds(
    p_fromAccount IN NUMBER,
    p_toAccount IN NUMBER,
    p_amount IN NUMBER
)
IS
    v_balance NUMBER;
BEGIN
    SELECT Balance
    INTO v_balance
    FROM Accounts
    WHERE AccountID = p_fromAccount;

    IF v_balance >= p_amount THEN

        UPDATE Accounts
        SET Balance = Balance - p_amount
        WHERE AccountID = p_fromAccount;

        UPDATE Accounts
        SET Balance = Balance + p_amount
        WHERE AccountID = p_toAccount;
```

```
            UPDATE Accounts
            SET Balance = Balance + p_amount
            WHERE AccountID = p_toAccount;

            COMMIT;

            DBMS_OUTPUT.PUT_LINE('Funds transferred successfully.');
        ELSE

            DBMS_OUTPUT.PUT_LINE('Insufficient balance.');

        END IF;
    END;
    /
    BEGIN
      TransferFunds(101,102,2000);
    END;
    /
```

Output :

```
SELECT * FROM Accounts;
```

Output:

```
 ACCOUNTID CUSTOMERNAME
---------- ----------------------------------------
ACCOUNTTYPE      BALANCE
-------------------- ----------
         101 John
Savings              10100

         102 Alice
Savings              15150

         103 David
Current              20000
```

```
SELECT * FROM Employees;
```

Output:

```
EMPLOYEEID EMPLOYEENAME
---------- ----------------------------------------
DEPARTMENT                     SALARY
------------------------------ ----------
         1 Rahul
IT                             55000

         2 Priya
HR                             40000

         3 Amit
IT                             66000
```

```
SELECT * FROM Accounts;
```

Output:

```
 ACCOUNTID CUSTOMERNAME
---------- --------------------------------
ACCOUNTTYPE      BALANCE
-------------------- ----------
       101 John
Savings             8201

       102 Alice
Savings          17301.5

       103 David
Current            20000
```